Spring 2026

Project on SystemC-TLM and OpenCV

Total points: 100

Section 1 - Objective (SystemC) – 60 pts:

To understand how to implement basic constructs of SystemC Transaction-Level Modeling (TLM-2.0) in system design. Model a data-driven system focusing on communication interfaces rather than pin-level signal toggling.

In this work, you are required to install SystemC on your system. There are two parts to this section.

Instructions:

You are compelled to use SystemC-TLM constructs.

Part A (15 pts): Basic Producer-Consumer via Blocking TLM

In this part, you will build a point-to-point communication link between a Traffic Generator and a Memory Sink.

1. The Producer (TPG - Test Pattern Generator)

- Function: Generate a synthetic image pattern (e.g., a color gradient or checkerboard) at a resolution of *780x610*.
- Requirement: The producer must act as a TLM Initiator. It should encapsulate pixel data into **tlm_generic_payload** and send it via **tlm_blocking_put_if** or a simple **b_transport** call.

2. The Consumer (Memory/Display)

- Function: TLM Target. It must allocate a memory buffer to store the incoming pixels.
- Requirement: Upon receiving the full frame, the consumer should export the data to a standard format (like a .ppm or .bmp file) to verify visual correctness.

Part B (45 pts) : Router Architecture

In this part, introduce an interconnect and basic timing annotation to observe how "bus" contention affects throughput.

1. The Interconnect (Router/Bus)

- Students must implement a simple **TLM Router** that sits between the Producer and Consumer.
- The Router should decode addresses to direct traffic to the Memory.

2. Timing Annotation

- Instead of untimed modeling, you must use sc_time to model delays.
- **Task:** Add a 10ns delay for every transaction at the Router and a 50ns write latency at the Memory. Calculate the total simulated time required to transfer one full frame (780 x 610 pixels).

Technical Requirements:

- Data Width: 32-bit (mapping RGB pixels).
- Interface: TLM-2.0 Base Protocol (Generic Payload).
- Tools: SystemC 2.3.x library, C++17, and a standard compiler (GCC/Clang).

Architecture:

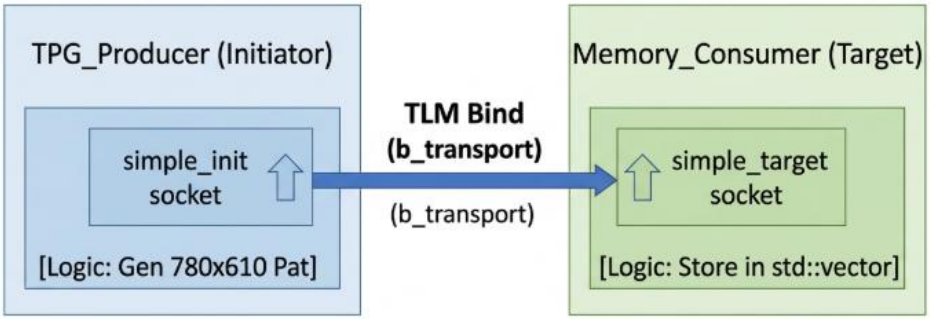

Figure 1. Part A: TPG-memory Architecture.

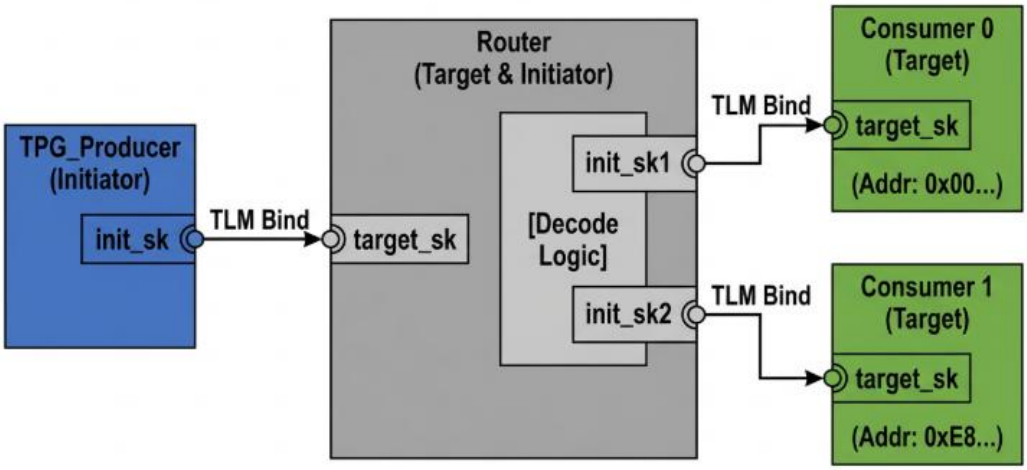

Figure 2. Part B: TPG-memory over router

Section 2 - Objective (SystemC + OpenCV) – 40 pts:

In this work, you are required to install SystemC in your system. You will be using OpenCV libraries along with SystemC library functions to implement some image processing applications.

Instructions:

Your goal in this homework is to apply the interface and channel concepts of SystemC to design an image processing system. OpenCV must be integrated into SystemC for this homework. Pictures will be read using OpenCV, processing in SystemC, and display with OpenCV.

In this work, you will be tasked to install OpenCV in your system. You will be using OpenCV library functions to implement some image processing applications.

We recommend using a 64-bit Linux system (It will be easier to link SystemC libraries with OpenCV libraries for future tasks). However, Visual Studio also works.

This site gives you a quick introduction into OpenCV

Task 1: OpenCV Installation (5 pts)

Follow the links below to download and install OpenCV Libraries on your system.

Installation instructions for Linux

Installation instructions for Windows

Implement a simple hello world program. The program should read a simple image (either from your webcam or the file enclosed and display the result. There are many tutorials (like this one) online you can use.

Task 1: Edge Detection (10 pts)

In this task, you will be writing a C++ code that will take an image file (.jpg) as input and perform edge detection. For edge detection, you will be using the OpenCV built-in libraries. (You can choose any edge detection algorithm as you please).

Task 2: OpenCV + SystemC-Acceleration (25 pts)

You are required to develop a line detection application. It will consist of two processors and a memory unit. The first processor (playing the role of the initiator) will provide the image that will be processed. The memory will serve as a temporary buffer. The second processor (playing the

role of the target) will read the content of the memory and use OpenCV functions to implement the line detection. Communication should be handled at the transaction level. The image resulting from the line detection will be stored as a file. The images below show an input image and an example of result obtained after running a line detection with OpenCV:

Fig2. Input Image

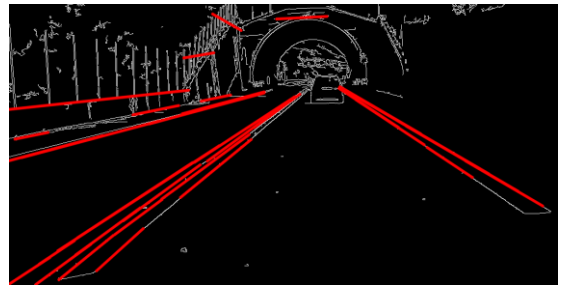

Fig3. Example of line detection

Note: You are only required to detect the lines on the streets. You do not need to identify specific lanes for this assignment. However, while drawing the detected lines on the output image try to avoid lines detected outside the road.

Submission:

Submit two zip files for each section. These files should be labelled **Student_names_section1** and **Student_names_section2**.

Section 1: Submit complete design to canvas.

Submit a single compressed archive (zip or .tar) containing your source code, build files and documentation.

1. **Source Code (The Hardware Models)**

Students must provide clean, commented C++ files:

- producer.h/cpp: The TPG module logic.
- consumer.h/cpp: The memory storage and PPM export logic.
- router.h/cpp: The address decoding logic (for Part B).
- sc_main.cpp: The top-level system where modules are instantiated and bound.

2. **Build & Verification Files**

- Makefile: Must be configured to link against SystemC and produce an executable named soc_sim.
- Generated Image: The final output_frame.ppm

3. The report

A page report of the following:

- In your own words, write a paragraph explaining your understanding of the system in part B.
- **Simulation Log:** A screenshot (showing your computer user) of the terminal output showing the simulation start/end times.
- **Analysis:** A brief explanation of how you handled the address offset for the second consumer in Part B.

Section 2:

Submit the following files (in a single zip file).

- <edge_StudentLastName.cpp> : The source code of the implementation.
- <outImage_StudentLastName.jpg> : The generated output image. Use the attached image of coins as input image.